C# PROGRAMMING

A Beginner’s Course

Learn the fundamentals of C# with clear explanations, practical examples, exercises, and a small console project.

LEVEL	Beginner
LANGUAGE	C# / .NET
FORMAT	Concepts + code + practice
SUGGESTED PACE	2–3 weeks at 1–2 hours per day

Designed as a foundation for further learning in .NET, ASP.NET Core, desktop applications, APIs, and software development.

How to use this course

Work through the chapters in order. Type every example yourself, run it, change values, and observe the result. Programming is learned by writing and debugging code—not only by reading it.

Learning outcomes

• Understand the C# language, .NET, and how a C# program runs.

• Write console applications using variables, data types, operators, and input/output.

• Use conditions, loops, methods, arrays, and collections.

• Understand object-oriented programming: classes, objects, inheritance, polymorphism, and encapsulation.

• Handle errors with exceptions and work with basic files.

• Build a small menu-driven console application.

What you need

• Visual Studio Community, or Visual Studio Code with the .NET SDK.

• A current supported .NET SDK installed on your computer.

• A terminal/command prompt for running .NET CLI commands.

Create and run your first project

```
dotnet --version
dotnet new console -n FirstCSharpApp
cd FirstCSharpApp
dotnet run
```

The template creates a console application. In modern .NET, the starter file may use top-level statements, so you might see Console.WriteLine(...) without an explicit class or Main method.

```
Console.WriteLine("Hello, C#!");
```

Output: **Hello, C#!**. Console.WriteLine prints text and moves to the next line. Console.Write prints without automatically moving to a new line.

1. Introduction to C# and .NET

C# (pronounced “C-sharp”) is a strongly typed, object-oriented, general-purpose programming language developed by Microsoft. It is commonly used for web APIs, enterprise software, desktop applications, cloud services, games, and tools.

C# versus .NET

• **C#** is the programming language used to write instructions.

• **.NET** is the development platform that provides a runtime, libraries, SDK tools, and application frameworks.

• **CLR** (Common Language Runtime) executes managed .NET code and provides services such as memory management and exception handling.

• **SDK** (Software Development Kit) includes tools to build, run, and publish applications.

A simplified execution flow is: C# source code → compiler → Intermediate Language (IL) → .NET runtime → execution on the operating system.

Comments

```
// Single-line comment
```

```
/*
   Multi-line comment
*/

/// <summary>
/// XML documentation comment for tools and IDEs.
/// </summary>
```

2. Variables and Data Types

A variable is a named storage location whose value can be used or changed while a program runs. C# is statically typed: a variable has a declared type.

```
int age = 22;
double price = 149.50;
decimal salary = 35000.75m;
char grade = 'A';
string name = "Avijit";
bool isActive = true;

Console.WriteLine(name);
Console.WriteLine(age);
```

Common built-in types

Type	Purpose	Example
int	Whole numbers	int count = 10;
long	Large whole numbers	long views = 9000000L;
float	Single-precision decimal	float rate = 2.5f;
double	Double-precision decimal	double pi = 3.14159;
decimal	High-precision decimal, often money	decimal total = 99.99m;
char	One character	char initial = 'A';
string	Text	string city = "Kolkata";
bool	true or false	bool passed = false;

Constants and type inference

```
const double TaxRate = 0.18; // cannot be reassigned
var message = "Type inferred as string";
var quantity = 5;             // inferred as int
```

Use **const** for compile-time constants. Use **var** when the compiler can clearly infer the type; var does not make the variable dynamically typed.

Nullable values

```
int? score = null; // nullable int
score = 95;

string? nickname = null; // nullable reference annotation
```

Nullable value types use ?. Nullable reference types help the compiler warn about possible null-reference errors when enabled in the project.

3. Input, Output, and Operators

Read user input

```
Console.Write("Enter your name: ");
string? name = Console.ReadLine();

Console.WriteLine($"Hello, {name}!");
```

Console.ReadLine returns text (string?) or null. Convert and validate input before using it as a number.

```
Console.Write("Enter your age: ");
string? input = Console.ReadLine();

if (int.TryParse(input, out int age))
{
    Console.WriteLine($"Next year you will be {age + 1}.");
}
else
{
    Console.WriteLine("Please enter a valid whole number.");
}
```

Operators

• **Arithmetic:** +, -, *, /, % (remainder).

• **Comparison:** ==, !=, <, >, <=, >=.

• **Logical:** && (AND), || (OR), ! (NOT).

• **Assignment:** =, +=, -=, *=, /=.

• **Increment/decrement:** ++ and --.

```
int a = 10, b = 3;
Console.WriteLine(a / b);  // 3: integer division
Console.WriteLine(a % b);  // 1: remainder
Console.WriteLine((double)a / b); // 3.333...
```

When both operands of / are integers, C# performs integer division. Convert one operand to a floating-point type when you need a fractional result.

4. Decision Making

if, else if, else

```
int marks = 76;

if (marks >= 90)
{
    Console.WriteLine("Grade A");
}
else if (marks >= 60)
{
    Console.WriteLine("Grade B");
}
else
{
    Console.WriteLine("Needs improvement");
}
```

switch

```
string day = "Monday";

switch (day)
{
    case "Monday":
        Console.WriteLine("Start of the week");
        break;
    case "Friday":
        Console.WriteLine("Almost weekend");
        break;
    default:
        Console.WriteLine("Regular day");
        break;
}
```

Conditional operator

```
int age = 20;
string result = age >= 18 ? "Adult" : "Minor";
Console.WriteLine(result);
```

5. Loops

Loops repeat a block of code. Choose the loop that makes the stopping condition clear.

for loop

```
for (int i = 1; i <= 5; i++)
{
    Console.WriteLine(i);
}
```

while loop

```
int count = 1;
while (count <= 5)
{
    Console.WriteLine(count);
    count++;
}
```

do-while loop

```
int option;
do
{
    Console.Write("Enter 0 to exit: ");
    int.TryParse(Console.ReadLine(), out option);
}
while (option != 0);
```

foreach loop

```
string[] names = { "Asha", "Ravi", "Mina" };

foreach (string person in names)
{
    Console.WriteLine(person);
}
```

break exits the nearest loop or switch. **continue** skips to the next loop iteration.

6. Methods

A method is a reusable block of code that performs a task. Methods can accept parameters and return a value.

```
static int Add(int x, int y)
{
    return x + y;
}

int answer = Add(10, 20);
Console.WriteLine(answer); // 30
```

Void methods and optional parameters

```
static void Greet(string name)
{
    Console.WriteLine($"Welcome, {name}!");
}

static double CalculateArea(double radius, double pi = 3.14159)
{
    return pi * radius * radius;
}
```

```
Greet("Avijit");
Console.WriteLine(CalculateArea(2));
```

Method overloading

```
static int Multiply(int a, int b) => a * b;
static double Multiply(double a, double b) => a * b;
```

Overloading means using the same method name with different parameter signatures. The compiler selects the matching method.

7. Arrays and Collections

Arrays

```
int[] marks = { 80, 75, 92, 68 };

Console.WriteLine(marks[0]); // first element

for (int i = 0; i < marks.Length; i++)
{
    Console.WriteLine(marks[i]);
}
```

Array indexes start at 0. An array has a fixed length after it is created.

List

```
using System.Collections.Generic;

List<string> skills = new List<string>();
skills.Add("C#");
skills.Add("SQL");
skills.Add("Python");

skills.Remove("SQL");

foreach (string skill in skills)
{
    Console.WriteLine(skill);
}
```

List is a resizable collection. The generic type T specifies what kind of elements it stores.

Dictionary

```
var student = new Dictionary<string, int>
{
    ["Asha"] = 85,
    ["Ravi"] = 91
};

if (student.TryGetValue("Ravi", out int marks))
{
    Console.WriteLine(marks);
}
```

A dictionary stores key/value pairs. Keys must be unique.

8. Object-Oriented Programming (OOP)

OOP organizes software around objects that combine state (data) and behavior (methods). C# supports the major OOP principles.

Class and object

```
class Student
{
```

```
    public string Name { get; set; }
    public int Age { get; set; }

    public void Introduce()
    {
        Console.WriteLine($"I am {Name}, age {Age}.");
    }
}

// Creating and using an object:
Student student = new Student
{
    Name = "Avijit",
    Age = 22
};
student.Introduce();
```

A **class** is a blueprint. An **object** is an instance of that class. Properties expose data; methods define behavior.

Encapsulation

```
class BankAccount
{
    private decimal balance;

    public void Deposit(decimal amount)
    {
        if (amount > 0)
            balance += amount;
    }

    public decimal GetBalance()
    {
        return balance;
    }
}
```

Encapsulation protects internal state. The private field cannot be accessed directly from outside the class; public methods control how it changes.

Constructors

```
class Employee
{
    public string Name { get; }

    public Employee(string name)
    {
        Name = name;
    }
}

Employee emp = new Employee("Ravi");
```

A constructor runs when an object is created. It commonly initializes required data.

Inheritance

```
class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating");
    }
}

class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Barking");
    }
```

```
}

Dog dog = new Dog();
dog.Eat();  // inherited
dog.Bark();
```

Inheritance lets a derived class reuse and extend an accessible base-class implementation.

Polymorphism

```
class Shape
{
    public virtual double Area() => 0;
}

class Circle : Shape
{
    public double Radius { get; set; }

    public override double Area()
        => Math.PI * Radius * Radius;
}

Shape shape = new Circle { Radius = 3 };
Console.WriteLine(shape.Area());
```

With runtime polymorphism, a base-class reference can call an overridden method on a derived object.

Abstraction and interfaces

```
interface IPayable
{
    decimal CalculatePay();
}

class Contractor : IPayable
{
    public decimal Hours { get; set; }
    public decimal Rate { get; set; }

    public decimal CalculatePay() => Hours * Rate;
}
```

An interface defines a contract. A class that implements it provides the required members. Abstraction focuses on what an object can do rather than its internal implementation.

9. Exception Handling

An exception is an error condition that interrupts normal execution. Use try/catch to handle errors you can reasonably recover from, and finally for cleanup that must run.

```
try
{
    Console.Write("Enter a number: ");
    int number = int.Parse(Console.ReadLine()!);
    Console.WriteLine(100 / number);
}
catch (FormatException)
{
    Console.WriteLine("Input must be a whole number.");
}
catch (DivideByZeroException)
{
    Console.WriteLine("The number must not be zero.");
}
finally
{
    Console.WriteLine("Operation finished.");
}
```

Prefer TryParse for ordinary user input validation. Do not use exceptions as the normal path for expected invalid input. Avoid catching Exception broadly unless you can handle or log it appropriately.

10. Basic File Handling

The System.IO namespace provides APIs for reading and writing files. The following example writes text and then reads it back.

```
using System.IO;

string path = "notes.txt";

File.WriteAllText(path, "Learning C# file handling.");
string content = File.ReadAllText(path);

Console.WriteLine(content);
```

For larger files, use streaming APIs such as StreamReader and StreamWriter. Handle expected I/O errors (for example, missing files or permission problems) and be mindful of the file path and working directory.

11. LINQ Basics

LINQ (Language Integrated Query) lets you query collections using a consistent syntax. It is commonly used for filtering, sorting, and transforming data.

```
using System.Linq;

int[] numbers = { 2, 5, 8, 11, 14, 17 };

var evenNumbers = numbers
    .Where(n => n % 2 == 0)
    .OrderBy(n => n);

foreach (int number in evenNumbers)
{
    Console.WriteLine(number);
}
```

Common LINQ operators include Where (filter), Select (project), OrderBy (sort), FirstOrDefault (first/default), Any (test existence), and Count (count elements). Many LINQ queries use deferred execution: the query runs when it is enumerated.

12. Mini Project: Student Marks Manager

Build a simple console program that stores student names and marks, lists all records, and calculates the class average. This project combines variables, collections, loops, methods, and input validation.

Starter implementation

```
using System;
using System.Collections.Generic;
using System.Linq;

class Program
{
    static void Main()
    {
        var marks = new Dictionary<string, int>();

        while (true)
        {
            Console.WriteLine("\n1. Add student");
            Console.WriteLine("2. View students");
            Console.WriteLine("3. Show average");
            Console.WriteLine("0. Exit");
```

```
            Console.Write("Choose: ");

            string? choice = Console.ReadLine();

            if (choice == "0") break;

            switch (choice)
            {
                case "1":
                    AddStudent(marks);
                    break;
                case "2":
                    ViewStudents(marks);
                    break;
                case "3":
                    ShowAverage(marks);
                    break;
                default:
                    Console.WriteLine("Invalid option.");
                    break;
            }
        }
    }

    static void AddStudent(Dictionary<string, int> marks)
    {
        Console.Write("Student name: ");
        string? name = Console.ReadLine();

        Console.Write("Marks (0-100): ");
        string? input = Console.ReadLine();

        if (string.IsNullOrWhiteSpace(name) ||
            !int.TryParse(input, out int score) ||
            score < 0 || score > 100)
        {
            Console.WriteLine("Enter a valid name and marks.");
            return;
        }

        marks[name] = score;
        Console.WriteLine("Student saved.");
    }

    static void ViewStudents(Dictionary<string, int> marks)
    {
        if (marks.Count == 0)
        {
            Console.WriteLine("No students added.");
            return;
        }

        foreach (var item in marks)
            Console.WriteLine($"{item.Key}: {item.Value}");
    }

    static void ShowAverage(Dictionary<string, int> marks)
    {
        if (marks.Count == 0)
        {
            Console.WriteLine("No marks available.");
            return;
        }

        double average = marks.Values.Average();
        Console.WriteLine($"Average: {average:F2}");
    }
}
```

Note: this is an in-memory beginner project. Data is lost when the program closes. Extend it by saving records to a file, preventing duplicate names, and adding edit/delete operations.

13. Practice Exercises

Try these without looking at a solution first. Test normal inputs, boundary values, and invalid inputs.

1. Variables: Declare variables for a person's name, age, height, and whether they are enrolled. Print them.

2. Input and operators: Ask for two numbers and display their sum, difference, product, and quotient. Handle division by zero.

3. Conditions: Read a mark from 0 to 100 and print a grade using your own grade boundaries. Reject invalid marks.

4. Loops: Print the multiplication table of a number from 1 to 10.

5. Methods: Write a method that accepts a number and returns true if it is prime.

6. Arrays: Find the largest, smallest, and average values in an integer array.

7. Collections: Store five product names in a List, then search for a product entered by the user.

8. OOP: Create a Book class with title, author, and price properties, plus a method to display its details.

9. Exceptions: Ask the user for an integer and safely handle invalid input.

10. LINQ: Given a list of numbers, select values greater than 10, sort them, and print the result.

14. Suggested Learning Roadmap

Stage	Topics	Practice target
1. Fundamentals	Syntax, variables, types, operators, input/output	Small calculator
2. Control flow	if/switch, loops, methods	Number guessing game
3. Data handling	Arrays, List, Dictionary, LINQ	Marks and product manager
4. OOP	Classes, properties, constructors, interfaces, inheritance	Bank account model
5. Practical C#	Exceptions, files, debugging, basic testing	Persistent console app
6. Next step	.NET projects, ASP.NET Core, Web API, SQL, async/await	CRUD Web API

Important topics to learn next

• Generics and collections in more depth.

• Delegates, events, and lambda expressions.

• Asynchronous programming with async and await.

• Dependency injection, configuration, and logging in .NET.

• Unit testing and debugging with Visual Studio.

• ASP.NET Core MVC or Web API, Entity Framework Core, and SQL Server.

15. Quick Reference

Task	C# example
Print	`Console.WriteLine("Hello");`
Read input	`string? value = Console.ReadLine();`
Parse safely	`int.TryParse(value, out int number)`
Declare list	`List<int> values = new();`

Task	C# example
Loop	`foreach (var item in values) { ... }`
Define method	`static int Add(int a, int b) => a + b;`
Handle exception	`try { ... } catch (FormatException) { ... }`
Write file	`File.WriteAllText("data.txt", "Hello");`
Filter with LINQ	`values.Where(x => x > 10)`

Final Advice

Build a habit of writing code every day. Start with small programs, read compiler errors carefully, use a debugger, and refactor your code as you learn. Once the fundamentals feel comfortable, move to .NET application development and build projects that solve real problems.

Version note: C# and .NET evolve. This course teaches foundational syntax and concepts that apply broadly to modern C#. Check the official Microsoft Learn documentation for version-specific details and current installation guidance.